

HDOC Day-39

Variadic Weighted Average and Command-Line Integer Processing in C

Question 1 - Variadic weightedAverage()

Algorithm

1. Start and call weightedAverage() with count followed by value-weight pairs.
2. Inside weightedAverage(), reject a non-positive count.
3. Initialize a va_list using va_start().
4. For each pair, obtain value and weight using va_arg(..., double).
5. If any weight is negative, display an error and return an invalid result.
6. Accumulate weightedSum += value * weight and totalWeight += weight.
7. Call va_end(). If totalWeight is zero, display an error.
8. Return weightedSum / totalWeight and display the result.

Test Case Table

TC	Input	Expected Result
1	weightedAverage(3, 80.0,0.3, 75.0,0.2, 90.0,0.5)	Weighted average = 84.00
2	weightedAverage(2, 10.0,0.0, 20.0,0.0)	Error: total weight cannot be zero.
3	weightedAverage(2, 10.0,0.5, 20.0,-0.2)	Error: weights must be non-negative.
4	weightedAverage(1, 50.0,1.0)	Weighted average = 50.00

C Program - Question 1

```
#include <stdio.h>
#include <stdarg.h>
#include <math.h>

double weightedAverage(int count, ...)
{
    va_list args;
    double weightedSum = 0.0, totalWeight = 0.0;
    int i;
    if (count <= 0) { fprintf(stderr, "Error: count must be positive.\n"); return NAN; }
    va_start(args, count);
    for (i = 0; i < count; i++) {
        double value = va_arg(args, double);
        double weight = va_arg(args, double);
        if (weight < 0.0) { va_end(args); fprintf(stderr, "Error: weights must be non-negative.\n"); return
NAN; }
        weightedSum += value * weight;
        totalWeight += weight;
    }
    va_end(args);
    if (totalWeight == 0.0) { fprintf(stderr, "Error: total weight cannot be zero.\n"); return NAN; }
    return weightedSum / totalWeight;
}
```

```
int main(void)
{
    double result = weightedAverage(3, 80.0, 0.3, 75.0, 0.2, 90.0, 0.5);
    if (!isnan(result)) printf("Weighted average = %.2f\n", result);
    return 0;
}
```

Compilation / Run

```
gcc hdoc39_q1.c -o hdoc39_q1 -lm
./hdoc39_q1
```

Compilation and Output

```
(kali㉿kali)-[~/Desktop]
$ vi hdoc39_q1.c

(kali㉿kali)-[~/Desktop]
$ gcc hdoc39_q1.c -o hdoc39_q1 -lm

(kali㉿kali)-[~/Desktop]
$ ./hdoc39_q1
Weighted average = 84.00
```

Question 2 - Command-Line Integer Statistics

Algorithm

9. Read all command-line arguments after the program name.
10. Validate each argument using strtol(); ignore and report non-integer arguments.
11. Store valid integers in a dynamically allocated integer array.
12. If no valid integers remain, display a message and stop.
13. Sort valid integers in ascending order using a user-defined sorting function.
14. Display the sorted integers, minimum, maximum, average, and median.
15. Scan backward from the maximum to find the second-largest distinct value.
16. If no second distinct value exists, report that it is not available.

Test Case Table

TC	Command-Line Input	Expected Result
1	./program 12 5 abc 8 12 20 3	Ignore abc; sorted: 3 5 8 12 12 20; min 3; max 20; avg 10.00; median 10.00; second-largest distinct 12
2	./program 7	Sorted: 7; min=max=7; avg=median=7.00; second-largest distinct not available
3	./program 5 5 5	Sorted: 5 5 5; second-largest distinct not available
4	./program abc xyz	Ignore both; no valid integers provided

C Program - Question 2

```
#include <stdio.h>
```

```

#include <stdlib.h>
#include <errno.h>
#include <limits.h>

int isInteger(const char *s, int *value) {
    char *end; long v;
    errno = 0;
    if (s == NULL || *s == '\\0') return 0;
    v = strtol(s, &end, 10);
    if (errno == ERANGE || v < INT_MIN || v > INT_MAX || *end != '\\0') return 0;
    *value = (int)v; return 1;
}

void sortAscending(int a[], int n) {
    int i,j,t; for(i=0;i<n-1;i++) for(j=0;j<n-1-i;j++) if(a[j]>a[j+1]){t=a[j];a[j]=a[j+1];a[j+1]=t;}
}

double average(const int a[], int n){ long long sum=0; int i; for(i=0;i<n;i++) sum+=a[i]; return
(double)sum/n; }

double median(const int a[], int n){ if(n%2) return a[n/2]; return ((double)a[n/2-1]+a[n/2])/2.0; }

int secondLargestDistinct(const int a[], int n, int *out){ int i; if(n<2)return 0; for(i=n-2;i>=0;i--)
if(a[i]!=a[n-1]){*out=a[i];return 1;} return 0; }

int main(int argc, char *argv[]){
    int *a, n=0, i, v, second;
    if(argc<2){printf("No command-line arguments provided.\\n");return 0;}
    a=malloc((argc-1)*sizeof *a); if(!a){fprintf(stderr,"Memory allocation failed.\\n");return 1;}
    for(i=1;i<argc;i++){ if(isInteger(argv[i],&v)) a[n++]=v; else printf("Invalid argument ignored:
%s\\n",argv[i]); }
    if(n==0){printf("No valid integers provided.\\n");free(a);return 0;}
    sortAscending(a,n);
    printf("Valid integers in ascending order:"); for(i=0;i<n;i++)printf(" %d",a[i]); printf("\\n");
    printf("Minimum: %d\\n",a[0]); printf("Maximum: %d\\n",a[n-1]);
    printf("Average: %.2f\\n",average(a,n)); printf("Median: %.2f\\n",median(a,n));
    if(secondLargestDistinct(a,n,&second)) printf("Second-largest distinct value: %d\\n",second); else
printf("Second-largest distinct value: Not available\\n");
    free(a); return 0;
}

```

Compilation / Run

```

gcc hdoc39_q2.c -o hdoc39_q2 -lm
./program 12 5 abc 8 12 20 3

```

Invalid / Boundary Test Case

```

(kali㉿kali)-[~/Desktop]
$ vi hdoc39_q2.c

(kali㉿kali)-[~/Desktop]
$ gcc hdoc39_q2.c -o hdoc39_q2 -lm

(kali㉿kali)-[~/Desktop]
$ ./hdoc39_q2 12 5 abc 8 12 20 3
Invalid argument ignored: abc
Valid integers in ascending order: 3 5 8 12 12 20
Minimum: 3
Maximum: 20
Average: 10.00
Median: 10.00
Second-largest distinct value: 12

```